



Cyberscope

A *TAC Security* Company

Audit Report

Vaulted Metrics Reserve

March 2026

SHA256

effe14d9c0b451e5453a43e7778d5044de72db7f0905f7af22e6b33f61f21b4d

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	5
VaultedMetricsVM888ReserveCyberScope1.sol	5
Findings Breakdown	6
Diagnostics	7
DRR - Deposit Revert Restriction	9
Description	9
Recommendation	10
ITA - Incorrect Token Address	11
Description	11
Recommendation	11
ICAR - Incompatible Chainlink Automation Refuel	12
Description	12
Recommendation	13
SPOM - Spot Price Oracle Manipulation	14
Description	14
Recommendation	15
EIA - EMA Initialization Assumption	16
Description	16
Recommendation	17
CCR - Contract Centralization Risk	18
Description	18
Recommendation	18
EWSS - Emergency Withdraw Stale State	19
Description	19
Recommendation	20
FALR - Fixed Amount LINK Refuel	21
Description	21
Recommendation	22
IEST - Inaccurate ETH Spend Tracking	23
Description	23
Recommendation	24
MCM - Misleading Comment Messages	25
Description	25
Recommendation	25

MC - Missing Check	26
Description	26
Recommendation	27
MEE - Missing Events Emission	28
Description	28
Recommendation	28
PLPI - Potential Liquidity Provision Inadequacy	29
Description	29
Recommendation	30
PMRM - Potential Mocked Registry Manipulation	31
Description	31
Recommendation	32
PSU - Potential Subtraction Underflow	33
Description	33
Recommendation	33
RV - Randomization Vulnerability	34
Description	34
Recommendation	34
RRA - Redundant Repeated Approvals	35
Description	35
Recommendation	35
SBI - Shield Balance Inconsistency	36
Description	36
Recommendation	37
SDAM - Shield Deployment Amount Mismatch	38
Description	38
Recommendation	39
TUU - Time Units Usage	40
Description	40
Recommendation	40
OCTD - Transfers Contract's Tokens	41
Description	41
Recommendation	42
UTPD - Unverified Third Party Dependencies	43
Description	43
Recommendation	44
ZD - Zero Division	45
Description	45
Recommendation	45
L04 - Conformance to Solidity Naming Conventions	46
Description	46
Recommendation	47

L19 - Stable Compiler Version	48
Description	48
Recommendation	48
L20 - Succeeded Transfer Check	49
Description	49
Recommendation	49
Functions Analysis	50
Inheritance Graph	51
Flow Graph	52
Summary	53
Disclaimer	54
About Cyberscope	55

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit

27 Mar 2026

Source Files

Filename	SHA256
VaultedMetricsVM888ReserveCyberScope1.sol	effe14d9c0b451e5453a43e7778d5044de7 2db7f0905f7af22e6b33f61f21b4d
libs/ReentrancyGuard.sol	60fa17b6450f26843322fdcc8359bd9a753 2cdcb9e3c0fb619afe8dd50793182
libs/Ownable.sol	d455c5a51099c741a934bdb7c59a1358fa 780b0062dc9f39c79e019574032c52
libs/IERC20.sol	2a2af50b19ca5e4072b290067931f0e4a64 398d761c23e7a4f90b70cbe87f3bc
libs/AutomationCompatible.sol	c50ffa57150f585ca7664d8cd356641e13b 0d6ca2e346bf12197ce041c843aaf

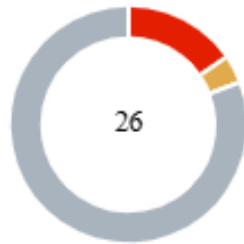
Overview

VaultedMetricsVM888ReserveCyberScope1.sol

VaultedMetrics Reserve is an autonomous buyback engine that acquires VM888 tokens on Uniswap V2 using deposited ETH, triggered every 12 hours via Chainlink Automation with a public `execute()` fallback. Each call executes two chamber beats: a FLOOR beat that buys VM888, adds liquidity, and burns LP tokens to 0xdead, plus an alternating REWARD (distributes tokens to staking/ecosystem wallets) or SHIELD (accumulates tokens for defensive LP deployment during dips).

Buy amounts are dynamically sized against an on-chain EMA across four market brackets mapped to hardcoded ETH ranges. The Shield deploys 25% as burned LP when price falls below EMA, with a 50% safety valve at 6M VM888. Deposits are restricted to whitelisted wallets: founders (one-time) and two treasury addresses (unlimited). All ecosystem addresses are immutable constants. A LINK auto-refuel mechanism maintains the Chainlink subscription every 30th call. After `renounceOwnership()`, all admin functions are permanently disabled and the contract operates autonomously until ETH is depleted.

Findings Breakdown



● Critical	4
● Medium	1
● Minor / Informative	21

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	4	0	0	0
● Medium	1	0	0	0
● Minor / Informative	21	0	0	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	DRR	Deposit Revert Restriction	Unresolved
●	ITA	Incorrect Token Address	Unresolved
●	ICAR	Incompatible Chainlink Automation Refuel	Unresolved
●	SPOM	Spot Price Oracle Manipulation	Unresolved
●	EIA	EMA Initialization Assumption	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	EWSS	Emergency Withdraw Stale State	Unresolved
●	FALR	Fixed Amount LINK Refuel	Unresolved
●	IENT	Inaccurate ETH Spend Tracking	Unresolved
●	MCM	Misleading Comment Messages	Unresolved
●	MC	Missing Check	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	PLPI	Potential Liquidity Provision Inadequacy	Unresolved
●	PMRM	Potential Mocked Registry Manipulation	Unresolved

●	PSU	Potential Subtraction Underflow	Unresolved
●	RV	Randomization Vulnerability	Unresolved
●	RRA	Redundant Repeated Approvals	Unresolved
●	SBI	Shield Balance Inconsistency	Unresolved
●	SDAM	Shield Deployment Amount Mismatch	Unresolved
●	TUU	Time Units Usage	Unresolved
●	OCTD	Transfers Contract's Tokens	Unresolved
●	UTPD	Unverified Third Party Dependencies	Unresolved
●	ZD	Zero Division	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved
●	L19	Stable Compiler Version	Unresolved
●	L20	Succeeded Transfer Check	Unresolved

DRR - Deposit Revert Restriction

Criticality	Critical
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L348,505,626
Status	Unresolved

Description

The contract's `receive` function routes all incoming ETH through a deposit whitelist validation. When the Uniswap Router returns excess ETH during liquidity operations, the refund triggers the `receive` function which reverts for non-whitelisted senders. This revert propagates through the liquidity call and is caught by the surrounding try/catch block, causing the contract to consistently fall back to direct token burns instead of creating liquidity pool positions.

```
Shell
receive() external payable {
    _processDeposit();
}

try UNISWAP_ROUTER.addLiquidityETH{value: lpHalf}(
    VM888_TOKEN, vm888ForLP, 0, 0, DEAD_ADDRESS,
    block.timestamp + 300
) {
} catch {
    IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,
    vm888ForLP);
}

try UNISWAP_ROUTER.addLiquidityETH{value:
ethForLP}(
    VM888_TOKEN, vm888Amount, 0, 0, DEAD_ADDRESS,
    block.timestamp + 300
) {
} catch {
```

```
IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,  
vm888Amount);  
}
```

Recommendation

The team is advised to decouple the deposit validation logic from the `receive` function, allowing the contract to accept ETH refunds from trusted protocol interactions without triggering whitelist restrictions.

ITA - Incorrect Token Address

Criticality	Critical
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L547
Status	Unresolved

Description

The contract queries the token balance by passing the recipient address as the token contract instead of the actual token address. Since the recipient is an EOA, the call returns empty data, causing an ABI decoding revert. This revert rolls back all state changes including the cycle counter, resulting in a permanent revert loop from the first execution that renders the entire contract non-functional.

```
Shell
uint256 vm888Before =
IERC20(recipient).balanceOf(recipient);
```

Recommendation

The team is advised to ensure that the correct token contract address is used when querying balances, referencing the intended token contract rather than the recipient address.

ICAR - Incompatible Chainlink Automation Refuel

Criticality	Critical
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L775
Status	Unresolved

Description

The `_checkAndRefuelLink` function is designed to automatically replenish the Chainlink Automation subscription with LINK tokens every 30th execution cycle. However, the `IAutomationRegistry` interface defined by the contract declares a `getSubscription` function with a return signature matching the Chainlink VRF Coordinator v2, not the Chainlink Automation Registry which uses `getUpkeep` with an entirely different return structure. When the function calls `getSubscription` on the actual Automation Registry, the call reverts because the function selector does not exist, and the surrounding try/catch silently catches the error and returns without performing any refueling.

```
Shell
try
IAutomationRegistry(chainlinkRegistry).getSubscript
ion(chainlinkSubId) returns (
    uint96 balance, uint64, address, address[]
memory
) {
    if (balance >= LINK_REFUEL_THRESHOLD) return;
} catch {
    return;
}
```

Recommendation

The team is advised to replace the `IAutomationRegistry` interface with the correct Chainlink Automation Registry interface that uses `getUpkeep(uint256 id)` and its corresponding return types, and to verify that the `addFunds` pattern aligns with the specific registry version deployed, as some versions require funding through LINK's `transferAndCall` (ERC-677) rather than the approve-and-call pattern currently implemented. This will ensure the auto-refuel mechanism functions as intended, maintaining the contract's long-term autonomous operation without manual LINK intervention.

SPOM - Spot Price Oracle Manipulation

Criticality	Critical
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L673,441,690,598
Status	Unresolved

Description

The contract derives its pricing from instantaneous Uniswap pair reserves, which can be manipulated within sandwich attacks or within the same block via flash loans. Since the spot price drives buy amount sizing, EMA updates, and Shield deployment triggers, a manipulator can distort the reserves and subsequently call the publicly accessible execution function to force unintended contract behavior.

Shell

```
function _getCurrentPrice() internal view returns
(uint256 price) {
    (uint112 reserve0, uint112 reserve1,) =
    UNISWAP_PAIR.getReserves();
    address token0 = UNISWAP_PAIR.token0();

    if (token0 == VM888_TOKEN) {
        price = (uint256(reserve1) * 1e18) /
uint256(reserve0);
    } else {
        price = (uint256(reserve0) * 1e18) /
uint256(reserve1);
    }
}

uint256 currentPrice = _getCurrentPrice();

function _getRandomBuyAmount(uint256 currentPrice,
uint256 salt) ...
```

```
else if (emaInitialized && currentPrice <  
emaPrice) {
```

Recommendation

The team is advised to implement a price derivation mechanism that is resistant to same block manipulation, ensuring that critical contract decisions are not based on values that can be externally influenced within a single transaction.

EIA - EMA Initialization Assumption

Criticality	Medium
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L441,444,598,697
Status	Unresolved

Description

The contract initializes its Exponential Moving Average from a single raw spot-price sample on the first execution. If the first call occurs during a period of manipulated reserves, the EMA baseline inherits the distorted value. With a low smoothing factor, the EMA requires a significant number of subsequent cycles to self-correct, during which all EMA-dependent decisions including buy sizing and Shield deployment operate on a biased reference point.

```
Shell
if (!emaInitialized) {
    emaPrice = currentPrice;
    emaInitialized = true;
}

uint256 currentPrice = _getCurrentPrice();

else if (currentPrice > emaPrice + (emaPrice *
RISING_THRESHOLD / BPS_DENOMINATOR)) {

else if (emaInitialized && currentPrice <
emaPrice) {
```

Recommendation

The team is advised to ensure that the initial reference value used for critical contract decisions is derived from a reliable and manipulation-resistant source, rather than a single observation that may not accurately reflect actual market conditions.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L306,317,129,356
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function setChainlinkConfig(uint256 _subId,  
address _registry) external onlyOwner  
function emergencyWithdrawETH() external onlyOwner  
nonReentrant  
function emergencyWithdrawTokens(address token)  
external onlyOwner nonReentrant  
  
function _processDeposit() internal
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

EWSS - Emergency Withdraw State State

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L329
Status	Unresolved

Description

The `emergencyWithdrawTokens` function allows the owner to withdraw the entire balance of any ERC20 token from the contract, including VM888 tokens that are tracked by the `shieldBalance` state variable. However, the function does not reset `shieldBalance` to zero when VM888 tokens are withdrawn. As a result, if the contract resumes operations after an emergency withdrawal, the `_executeShield` function will attempt to deploy tokens based on the stale `shieldBalance` value, and the subtraction `shieldBalance -= vm888Amount` in `_deployShieldAsLP` will underflow causing a revert.

Shell

```
function emergencyWithdrawTokens(address token)
external onlyOwner nonReentrant {
    uint256 balance =
IERC20(token).balanceOf(address(this));
    require(balance > 0, "No tokens to withdraw");
IERC20(token).transfer(msg.sender, balance);
    emit EmergencyWithdraw(token, balance);
}
```

Recommendation

The team is advised to update the `emergencyWithdrawTokens` function to check whether the withdrawn token is VM888 and, if so, reset `shieldBalance`. This will ensure internal accounting remains consistent with the actual token balance held by the contract, preventing potential denial-of-service conditions if operations resume after an emergency withdrawal.

FALR - Fixed Amount LINK Refuel

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L473,619
Status	Unresolved

Description

The `_checkAndRefuelLink` function uses a fixed ETH amount of `BUY_STABLE_MIN` (0.000072 ETH) to purchase LINK tokens for subscription refueling, regardless of the current LINK market price. Additionally, the refuel is only attempted every 30th execution cycle (approximately every 15 days), with no mechanism to adjust frequency based on actual LINK consumption or balance urgency. If the price of LINK increases significantly, the fixed ETH will purchase a negligible amount of LINK that may be insufficient to fund even a single Chainlink Automation call, rendering the refuel effectively useless despite executing successfully. Conversely, the 15-day gap between refuel checks means a rapidly depleting subscription could run out of LINK well before the next scheduled check, causing automation to halt. As a result, the refueling mechanism lacks the adaptability needed to maintain reliable automation under changing market conditions, and a sustained increase in LINK price could permanently stall the contract's autonomous execution.

```
Shell
if (totalCalls % REFUEL_FREQUENCY == 0) {
    _checkAndRefuelLink();
}
...

uint256 ethForLP = BUY_STABLE_MIN;
```

Recommendation

The team is advised to implement a dynamic ETH allocation for LINK purchases that scales based on the current LINK/ETH price ratio or targets a minimum LINK output amount rather than spending a fixed ETH input, and to consider checking the subscription balance on every execution cycle rather than every 30th call so that critically low balances are addressed promptly. This will ensure the contract can sustain its Chainlink Automation regardless of LINK price fluctuations and avoid unnecessary downtime between refuel windows.

IENT - Inaccurate ETH Spend Tracking

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L514
Status	Unresolved

Description

The `_execute` functions records the full amount in the `totalEthSpent` state variable after executing the token swap and liquidity addition. However, the `addLiquidityETH` call is wrapped in a try/catch block, and when it reverts the amount is returned to the contract since failed external calls in try/catch do not transfer value. Despite this, `totalEthSpent` still increments by the full amount, causing the contract to overcount the ETH it has actually spent. This is also true for the amounts returned from the pair in a successful liquidity addition. Furthermore in case the liquidity addition is skipped entirely the `totalEthSpent` will still be updated.

Shell

```
try UNISWAP_ROUTER.addLiquidityETH{value: lpHalf}{
    VM888_TOKEN,
    vm888ForLP,
    0,
    0,
    DEAD_ADDRESS,
    block.timestamp + 300
} {
} catch {
    IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,
    vm888ForLP);
}
...

totalEthSpent += buyAmount;
```

Recommendation

The team is advised to only update the `totalEthSpent` by the actual amount used for the operations.

MCM - Misleading Comment Messages

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L497
Status	Unresolved

Description

The contract is using misleading comment messages. These comment messages do not accurately reflect the actual implementation, making it difficult to understand the source code. As a result, the users will not comprehend the source code's actual implementation.

```
Shell
// Subtract any shield balance from available
VM888 for LP

uint256 vm888ForLP = vm888Bought;
```

Recommendation

The team is advised to carefully review the comment in order to reflect the actual implementation. To improve code readability, the team should use more specific and descriptive comment messages.

MC - Missing Check

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L356,657,767
Status	Unresolved

Description

The `_processDeposit` function does not validate that `msg.value` is greater than zero. The founder's wallet is restricted to a single deposit via the `founderHasDeposited` flag, which is permanently set to `true` regardless of the amount. If the founders wallet calls the function with zero value, the one-time deposit opportunity is irreversibly consumed without any ETH being deposited

Shell

```
function _processDeposit() internal {
    if (msg.sender == FOUNDER_WALLET) {
        require(!founderHasDeposited, "Founders:
one-time deposit already made");
        founderHasDeposited = true;

        ...
    }
}
```

Additionally, the contract does not calculate or enforce a minimum output amount on swap operations. The `amountOutMin` parameter is hardcoded to zero on all swap calls, accepting any output regardless of price impact. While the swap amounts are bounded by small hardcoded constants, the absence of slippage validation deviates from best practices and leaves the operations without price protection.

```
Shell
UNISWAP_ROUTER.swapExactETHForTokensSupportingFeeOnT
ransferTokens(value: ethAmount){
    0,
    path,
    recipient,
    block.timestamp + 300
};

try
UNISWAP_ROUTER.swapExactETHForTokensSupportingFeeOnT
ransferTokens(value: ethForLink){
    0,
    path,
    address(this),
    block.timestamp + 300
};
```

Recommendation

The team is advised to include proper validation checks to ensure that the processed values meet the required specifications before executing state-changing operations.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L306
Status	Unresolved

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

```
Shell
function setChainlinkConfig(uint256 _subId,
address _registry) external onlyOwner {
    ...
    chainlinkSubId = _subId;
    chainlinkRegistry = _registry;
}
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

PLPI - Potential Liquidity Provision Inadequacy

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L494,550,579,657
Status	Unresolved

Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

Shell

```
function _swapETHForVM888(uint256 ethAmount, address
recipient) internal {

    UNISWAP_ROUTER.swapExactETHForTokensSupportingFeeOnT
ransferTokens{value: ethAmount}(
        0, path, recipient, block.timestamp + 300
    );
}

_swapETHForVM888(buyHalf, address(this));

_swapETHForVM888(buyAmount, recipient);

_swapETHForVM888(buyAmount, address(this));
```

Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

PMRM - Potential Mocked Registry Manipulation

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L306
Status	Unresolved

Description

The contract includes a method that allows the owner to modify the `chainlinkRegistry` address. While this feature provides flexibility, it introduces a security threat. The owner could set the `chainlinkRegistry` address to any contract that implements the interface, potentially containing malicious code.

```
Shell
function setChainlinkConfig(uint256 _subId,
address _registry) external onlyOwner {
    require(_subId > 0, "Invalid subscription
ID");
    require(_registry != address(0), "Invalid
registry address");
    chainlinkSubId = _subId;
    chainlinkRegistry = _registry;
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

PSU - Potential Subtraction Underflow

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L492,547,582,788
Status	Unresolved

Description

The contract subtracts two values, the second value may be greater than the first value if the contract owner misuses the configuration. As a result, the subtraction may underflow and cause the execution to revert.

```
Shell
address(this).balance - buyAmount <
MIN_ETH_RESERVE

address(this).balance - ethForLink <
MIN_ETH_RESERVE
```

Recommendation

The team is advised to properly handle the code to avoid underflow subtractions and ensure the reliability and safety of the contract. The contract should ensure that the first value is always greater than the second value. It should add a sanity check in the setters of the variable or not allow executing the corresponding section if the condition is violated.

RV - Randomization Vulnerability

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L712,718
Status	Unresolved

Description

The contract is using an on-chain technique in order to determine random numbers. The blockchain runtime environment is fully deterministic, as a result, the pseudo-random numbers could be predicted.

```
Shell
uint256 randomSeed =
uint256(keccak256(abi.encodePacked(
    block.timestamp,
    block.prevrandao,
    beatCount,
    currentPrice,
    salt
)));
```

Recommendation

The contract could use an advanced randomization technique that guarantees an acceptable randomization factor. For instance, the Chainlink VRF (Verifiable Random Function). <https://docs.chain.link/docs/chainlink-vrf>

RRA - Redundant Repeated Approvals

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L511,637
Status	Unresolved

Description

The contract is designed to `approve` token transfers during the contract's operation by calling the `_approve` function before specific operations. This approach results in additional gas costs since the approval process is repeated for every operation execution, leading to inefficiencies and increased transaction expenses.

```
Shell
IERC20(VM888_TOKEN).approve(address(UNISWAP_ROUTER), vm888ForLP);

IERC20(VM888_TOKEN).approve(address(UNISWAP_ROUTER), vm888Amount);
```

Recommendation

Since the approved address is a trusted third-party source, it is recommended to optimize the contract by approving the maximum amount of tokens once in the initial set of the variable, rather than before each operation. This change will reduce the overall gas consumption and improve the efficiency of the contract.

SBI - Shield Balance Inconsistency

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L613,624,626
Status	Unresolved

Description

The contract decrements the `shieldBalance` counter by the full deployment amount before attempting to add liquidity. The `addLiquidityETH` function pairs tokens at the current pool ratio and may not consume the entire token amount provided. Any excess tokens returned to the contract are not added back to the `shieldBalance` counter, resulting in accounting drift where the contract holds more tokens than the internal counter reflects.

Shell

```
shieldBalance -= vm888Amount;

IERC20(VM888_TOKEN).approve(address(UNISWAP_ROUTER), vm888Amount);

try UNISWAP_ROUTER.addLiquidityETH{value:
ethForLP}(
    VM888_TOKEN,
    vm888Amount,
    0,
    0,
    DEAD_ADDRESS,
    block.timestamp + 300
) {
```

Recommendation

The team is advised to ensure that internal accounting variables accurately reflect the actual state of the contract's holdings after executing external operations that may partially consume the allocated amounts.

SDAM - Shield Deployment Amount Mismatch

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L619
Status	Unresolved

Description

The `_deployShieldAsLP` function pairs the entire accumulated `shieldBalance` with a fixed ETH amount of `BUY_STABLE_MIN` (0.000072 ETH). Uniswap's `addLiquidityETH` only consumes tokens proportional to the provided ETH based on the current pool ratio, meaning the vast majority of the VM888 tokens will not be used in the liquidity pairing. However, the function decrements `shieldBalance` by the full `vm888Amount` before the liquidity call, leaving the unconsumed tokens in the contract with no state variable tracking them.

```
Shell
function _deployShieldAsLP(uint256 vm888Amount,
string memory reason) internal {
    if (vm888Amount == 0 || address(this).balance
<= MIN_ETH_RESERVE) return;
    shieldBalance -= vm888Amount;
    uint256 ethForLP = BUY_STABLE_MIN;
    if (address(this).balance - ethForLP <
MIN_ETH_RESERVE) {
        IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,
vm888Amount);
        emit ShieldDeployed(beatCount,
vm888Amount, reason);
        return;
    }
    IERC20(VM888_TOKEN).approve(address(UNISWAP_ROUTER
), vm888Amount);
    try UNISWAP_ROUTER.addLiquidityETH{value:
ethForLP}(VM888_TOKEN, vm888Amount, 0, 0, DEAD_ADDRESS
, block.timestamp + 300
    ) {
    } catch {
        IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,
vm888Amount);
    }
    emit ShieldDeployed(beatCount, vm888Amount,
reason);
}
```

Recommendation

The team is advised to calculate the ETH pairing amount proportionally based on the current pool ratio and the VM888 amount being deployed, or alternatively to only decrement `shieldBalance` by the amount of tokens actually consumed by the router rather than the full requested amount. This will ensure that the Shield mechanism functions as designed and that no tokens are silently orphaned within the contract.

TUU - Time Units Usage

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L511,631,661,771
Status	Unresolved

Description

The contract is using arbitrary numbers to form time-related values. As a result, it decreases the readability of the codebase and prevents the compiler from optimizing the source code.

Shell

```
block.timestamp + 300
```

Recommendation

It is good practice to use the time units reserved keywords like `seconds`, `minutes`, `hours`, `days` and `weeks` to process time-related calculations.

It's important to note that these time units are simply a shorthand notation for representing time in seconds, and do not have any effect on the actual passage of time or the execution of the contract. The time units are simply a convenience for expressing time in a more human-readable form.

OCTD - Transfers Contract's Tokens

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L317,329
Status	Unresolved

Description

The contract owner has the authority to claim all the balance of the contract. The owner may take advantage of it by calling the `emergencyWithdrawETH` and `emergencyWithdrawTokens` functions.

Additionally due to the restrictions of the `_processDeposit` function it may be impossible to re-add the eth withdrawn.

Shell

```
function emergencyWithdrawETH() external onlyOwner
nonReentrant {
    uint256 balance = address(this).balance;
    require(balance > 0, "No ETH to withdraw");
    (bool success, ) = msg.sender.call{value:
balance}("");
    require(success, "ETH transfer failed");
    emit EmergencyWithdraw(address(0), balance);
}

function emergencyWithdrawTokens(address token)
external onlyOwner nonReentrant {
    uint256 balance =
IERC20(token).balanceOf(address(this));
    require(balance > 0, "No tokens to withdraw");
    IERC20(token).transfer(msg.sender, balance);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

UTPD - Unverified Third Party Dependencies

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L306,746,779
Status	Unresolved

Description

The contract uses an external contract in order to determine the transaction's flow. The external contract is untrusted. As a result, it may produce security issues and harm the transactions.

Shell

```
function setChainlinkConfig(uint256 _subId,
address _registry) external onlyOwner {
    require(_subId > 0, "Invalid subscription ID");
    require(_registry != address(0), "Invalid
registry address");
    chainlinkSubId = _subId;
    chainlinkRegistry = _registry;
}

try
IAutomationRegistry(chainlinkRegistry).getSubscript
ion(chainlinkSubId) returns ( uint96 balance,
uint64, address, address[] memory)

try
IAutomationRegistry(chainlinkRegistry).addFunds(cha
inlinkSubId, uint96(linkReceived)) {
    emit LinkRefueled(linkReceived);
}
```

Recommendation

The contract should use a trusted external source. A trusted source could be either a commonly recognized or an audited contract. The pointing addresses should not be able to change after the initialization.

ZD - Zero Division

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L674,681
Status	Unresolved

Description

The contract is using variables that may be set to zero as denominators. This can lead to unpredictable and potentially harmful results, such as a transaction revert.

```
Shell
function _getCurrentPrice() internal view returns
(uint256 price) {
    (uint112 reserve0, uint112 reserve1,) =
    UNISWAP_PAIR.getReserves();
    ...

    price = (uint256(reserve1) * 1e18) /
    uint256(reserve0);
}
}
```

Recommendation

It is important to handle division by zero appropriately in the code to avoid unintended behavior and to ensure the reliability and safety of the contract. The contract should ensure that the divisor is always non-zero before performing a division operation. It should prevent the variables to be set to zero, or should not allow the execution of the corresponding statements.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L98,306
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

Shell

```
function WETH() external pure returns (address);

function setChainlinkConfig(uint256 _subId,
address _registry) external onlyOwner {
    ...
    _subId
    _registry
}
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L2
Status	Unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

Shell

```
pragma solidity ^0.8.19;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

L20 - Succeeded Transfer Check

Criticality	Minor / Informative
Location	VaultedMetricsVM888ReserveCyberScope1.sol#L332,516,619,637
Status	Unresolved

Description

According to the ERC20 specification, the transfer methods should be checked if the result is successful. Otherwise, the contract may wrongly assume that the transfer has been established.

Shell

```
IERC20(token).transfer(msg.sender, balance);
```

```
IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,  
vm888ForLP);
```

```
IERC20(VM888_TOKEN).transfer(DEAD_ADDRESS,  
vm888Amount);
```

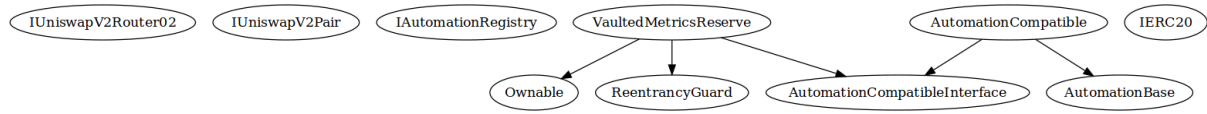
Recommendation

The contract should check if the result of the transfer methods is successful. The team is advised to check the SafeERC20 library from the [Openzeppelin library](#).

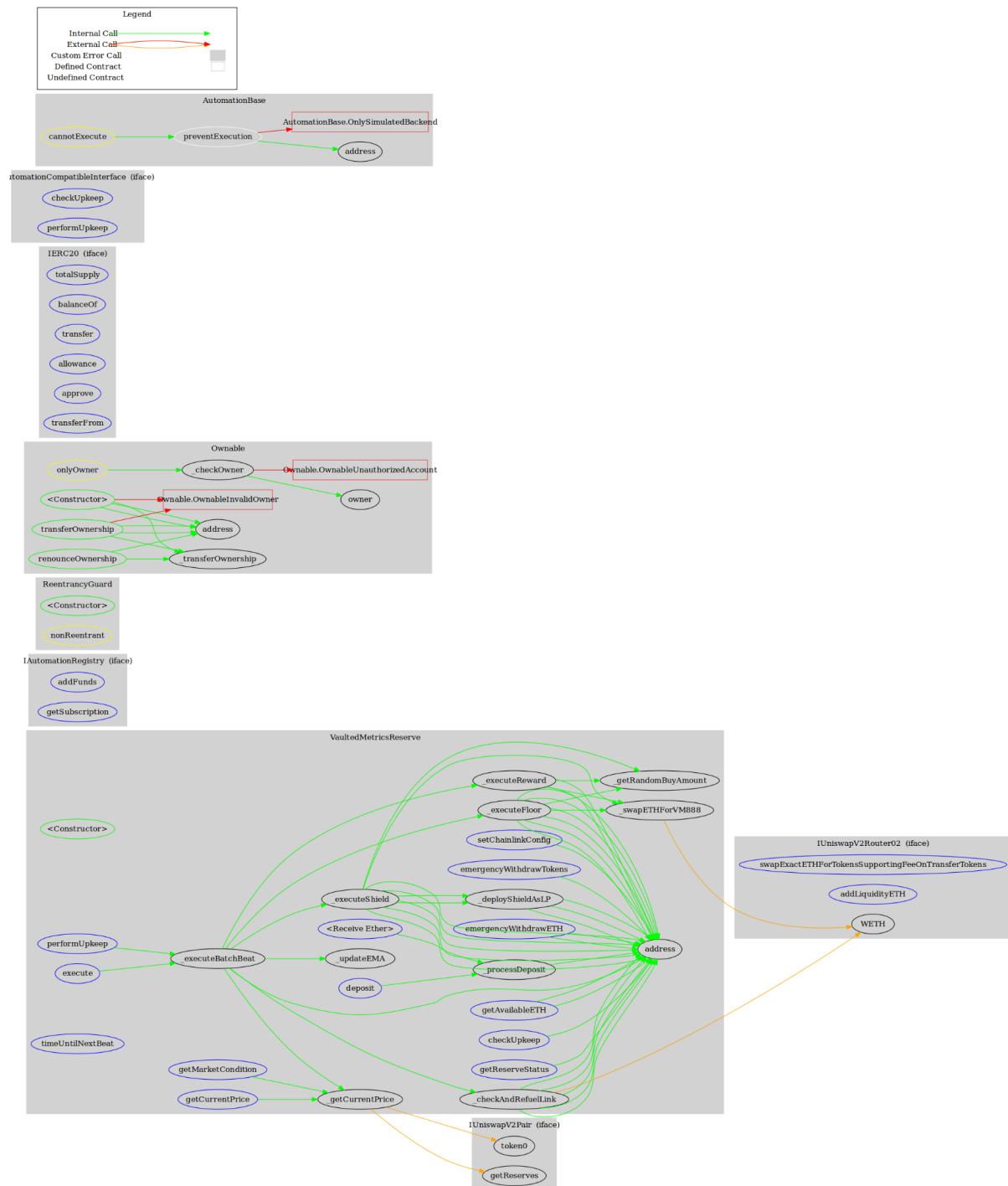
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
IUniswapV2Router02	Interface			
	swapExactETHForTokensSupportingFeeOnTransferTokens	External	Payable	-
	addLiquidityETH	External	Payable	-
	WETH	External		-
IUniswapV2Pair	Interface			
	getReserves	External		-
	token0	External		-
IAutomationRegistry	Interface			
	addFunds	External	✓	-
	getSubscription	External		-
AutomationCompatibleInterface	Interface			
	checkUpkeep	External	✓	-
	performUpkeep	External	✓	-

Inheritance Graph



Flow Graph



Summary

VaultedMetrics Reserve contract implements an autonomous buyback mechanism for the VM888 token ecosystem. This audit investigates security issues, business logic concerns and potential improvements. The Smart Contract analysis reported one critical issue related to an incorrect token address reference that prevents contract execution, along with medium-severity findings affecting liquidity operations and price derivation. The remaining findings are minor observations related to best practices and code quality.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io